

RAPPORT TECHNIQUE : PIPELINE INTER-AGENTS

Flux de Données : Extractor Agent → Agent Analyseur

1. Architecture du Flux (Asynchrone)

Pour garantir la performance et éviter les blocages, la communication est strictement **asynchrone**. L'Extractor pousse les événements nettoyés dans la **Shared Memory**, l'Analyseur les consomme en boucle continue via un thread dédié (*Polling Loop*).

2. Canaux de Communication (Shared Memory)

L'Agent Analyseur doit écouter activement deux canaux distincts :

- **events_structured** : Flux global de tous les événements normalisés (Réseau et Syslog).
- **pending_analysis** : Flux prioritaire contenant uniquement les événements suspects (Sévérité ≥ 0.7 ou mots-clés d'attaque).

3. Format Standardisé des Données (Payload JSON)

Chaque événement envoyé est sérialisé sous forme de dictionnaire JSON "flat". L'Analyseur reçoit **strictement** cette structure :

Listing 1: Format de l'événement normalisé

```
{
  "event_id": "8a3f2b1c",           // ID unique généré selon le contenu
  "timestamp": "2026-06-17T19:09:29", // Format ISO 8601 UTC
  "source": "syslog",              // Origine : "syslog" ou "network"
  "severity": 0.50,                // Score normalisé entre 0.0 et 1.0
  "message": "Failed password for root from 192.168.1.105 port 22 ssh2",
  "raw": "<13>Jun 17 19:09:29 sshd[123]: Failed password for root..."
}
```

4. Attentes et Logique de l'Agent Analyseur

Une fois les données récupérées, l'Analyseur doit exécuter 4 tâches principales :

1. **Analyse Parsing** : Extraire les entités critiques du champ **message** (ex: IPs sources, utilisateurs, ports).
2. **Détection d'Anomalies** : Identifier les comportements suspects via des règles heuristiques (ex: seuil d'échecs).
3. **Corrélation Temporelle** : Croiser les sources. *Exemple* : Si la même IP génère un scan de port (**source: network**) ET des échecs d'authentification (**source: syslog**), déclencher une alerte globale.
4. **Classification** : Rehausser la criticité de l'incident au niveau maximal (**CRITICAL**) en cas de corrélation positive.

5. Code d'Intégration Minimal (Python)

```
from src.utils.shared_memory import SharedMemory

memory = SharedMemory()
while True:
    events = memory.read("events_structured") # boucle active
    if events:
        for event in events:
            # Traitement de l'analyseur ici
            print(f"Analyse de l'événement {event['event_id']} (Source: {event['source']})")
```